Tic-Tac-Toe

Projektdokumentation

Java Swing • GUI-Anwendung • Modulare Architektur

Version 1.0 • Mai 2026 • Java 17+

Autoren: Fabio Stoffels, Gabriel de Souza leal, Peter Kretzer

Inhaltsverzeichnis

Inhaltsverzeichnis 2
1. Projektüberblick 3
1.1 Ziel des Projekts 3
1.2 Technische Rahmenbedingungen 3
1.3 Versionierung des Projekts 3
2. Projektstruktur 3
2.1 Ordnerstruktur 4
3. Architektur 4
3.1 Schichtenmodell 4
3.2 Kommunikationsfluss 4
3.3 Design-Entscheidungen 5
4. Klassen-Referenz 5
4.1 PlayerSymbol (Enum) 5
4.2 GameLogic 5
4.3 GameBoard 6
4.4 GameController 6
4.5 GameWindow 6
5. Spielablauf 7
5.1 Initialisierung 7
5.2 Spielzug 7
5.3 Gewinnerprüfung 7
5.4 Neustart 7
6. Kompilierung & Start 7
6.1 Voraussetzungen 8
6.2 Terminal-Befehle 8
6.3 Bekannte Einschränkungen 8
7. Erweiterungsmöglichkeiten 8
7.1 KI-Implementierung (Skizze) 9
8. Codequalität & Konventionen 9
8.1 Kommentarsprache 9
8.2 JavaDoc 9
8.3 OOP-Prinzipien 10
9. Vollständige Datei-Übersicht 10

1. Projektüberblick

Dieses Dokument beschreibt die Architektur, den Aufbau und die Funktionsweise des Tic-Tac-Toe-Projekts, das als Java-Anwendung mit grafischer Benutzeroberfläche (GUI) auf Basis von Swing umgesetzt wurde.

Auf einen Blick	
Sprache:	Java 17+
GUI-Bibliothek:	Java Swing (keine externen Abhängigkeiten)
Architektur:	Mehrschichtig – Logik, GUI und Controller getrennt
Dateien:	5 .java-Quelldateien
Designthema:	Catppuccin (dunkles Farbschema)

1.1 Ziel des Projekts

Das Projekt implementiert das klassische Zwei-Spieler-Brettspiel Tic-Tac-Toe als eigenständige Java-Desktop-Anwendung. Ziel war neben der Funktionalität auch eine saubere, objektorientierte Codestruktur mit klarer Trennung von Spiellogik und Präsentation.

1.2 Technische Rahmenbedingungen

- Java SE 17 oder höher (kompatibel mit Java 21 LTS)
- Ausschließlich Standard-Java-Bibliotheken (kein Maven, kein Gradle, keine Drittanbieter)
- Startbar mit: javac *.java && java Main
- Getestet unter Windows, macOS und Linux

1.3 Versionierung des Projekts

- Die Versionierung und somit auch die Speicherung läuft über GitHub, hier wird über Branche‘s das Projekt Stück für Stück aufgebaut.
- Link zum Repository: https://github.com/D-PEKR/Java_Repo

2. Projektstruktur

Das Projekt ist in fünf Quelldateien aufgeteilt, von denen jede eine klar abgegrenzte Aufgabe übernimmt. Diese Aufteilung folgt dem Prinzip der Separation of Concerns.

Datei	Zweck	Abhängigkeiten
Main.java	Einstiegspunkt, startet die GUI-Anwendung	GameWindow
PlayerSymbol.java	Enum für X, O und LEER	(keine)
GameLogic.java	Spielzustand, Züge, Gewinner-/Unentschiedenprüfung	PlayerSymbol
GameBoard.java	Swing-Panel mit 3×3-Buttons, Hover-Effekte	GameLogic, PlayerSymbol
GameController.java	ActionListener – verbindet GUI und Logik	GameLogic, GameBoard, GameWindow
GameWindow.java	JFrame – Hauptfenster mit Titel, Status, Neustart	GameBoard, GameController, PlayerSymbol

2.1 Ordnerstruktur

```
TicTacToe/
  Main.java           <- Programm-Einstieg
  PlayerSymbol.java   <- Enum (X | O | LEER)
  GameLogic.java      <- Spielregeln & Zustand
  GameBoard.java      <- Swing-Spielfeld (GUI)
  GameController.java <- Steuerlogik (MVC-Controller)
  GameWindow.java     <- Hauptfenster (MVC-View)
```

3. Architektur

Das Projekt folgt einer angepassten MVC-Architektur (Model–View–Controller). Die drei Schichten kommunizieren ausschließlich über klar definierte Methoden-Schnittstellen.

3.1 Schichtenmodell

Schicht	Klasse(n)	Beschreibung
Model	GameLogic, PlayerSymbol	Verwaltet den Spielzustand, kennt kein Swing
View	GameBoard, GameWindow	Zeigt den Zustand an, enthält keine Spielregeln
Controller	GameController	Verbindet Model und View, reagiert auf Klicks

3.2 Kommunikationsfluss

Der Ablauf eines einzelnen Spielzugs ist wie folgt:

1. Spieler klickt auf einen Button im GameBoard.
2. GameBoard löst ein ActionEvent mit ActionCommand "zeile,spalte" aus.
3. GameController empfängt das Event (actionPerformed).
4. GameController ruft GameLogic.zugAusfuehren(zeile, spalte) auf.
5. GameLogic prüft den Zug, aktualisiert das Brett und erkennt ggf. Spielende.
6. GameController ruft GameBoard.aktualisiereButton() auf.
7. Falls Spielende: GameController ruft GameWindow.zeigeSpielEnde() auf.

3.3 Design-Entscheidungen

- GameLogic hat keine Swing-Importe – sie kann isoliert getestet werden.
- GameBoard kennt keine Spielregeln – es ist ein reines Anzeige-Panel.
- Der ActionCommand codiert die Position als "zeile,spalte"-String, was die Buttons generisch hält.
- Das Hover-Verhalten wird per MouseAdapter simuliert, da Swing kein natives CSS-Hover kennt.

4. Klassen-Referenz

4.1 PlayerSymbol (Enum)

Aufzählungstyp für die drei möglichen Zustände einer Spielfeldzelle.

Konstante	Textdarstellung	Bedeutung
LEER	" "	Noch kein Zug auf dieser Zelle
X	"X"	Zelle belegt von Spieler X
O	"O"	Zelle belegt von Spieler O

Wichtige Methode: `andererSpieler()` – gibt das Symbol des Gegenspielers zurück (X→O, O→X).

4.2 GameLogic

Die Kernklasse für Spielzustand und Regelprüfung. Keine GUI-Abhängigkeit.

Methode	Rückgabetyp	Beschreibung
spielZuruecksetzen()	void	Setzt Brett und Spielerzustand auf Ausgangswert zurück
zugAusfuehren(zeile, spalte)	boolean	Setzt Symbol auf Zelle; false bei ungültigem Zug
pruefeGewinner(symbol)	boolean	Prüft alle Zeilen, Spalten und Diagonalen

Methode	Rückgabetyp	Beschreibung
istUnentschieden()	boolean	True wenn alle Felder belegt und kein Gewinner
getZelle(zeile, spalte)	PlayerSymbol	Gibt das Symbol der angegebenen Zelle zurück
getAktuellerSpieler()	PlayerSymbol	Gibt den gerade aktiven Spieler zurück
istSpielBeendet()	boolean	True nach Sieg oder Unentschieden

4.3 GameBoard

Swing-Panel (extends JPanel) mit einem GridLayout(3,3,8,8) aus 9 JButtons.

Methode	Beschreibung
setzeButtonListener(listener)	Registriert ActionListener an allen 9 Buttons
aktualisiereButton(zeile, spalte, symbol)	Setzt Text und Farbe eines Buttons
felderZuruecksetzen()	Setzt alle 9 Buttons auf LEER zurück
alleButtonsDeaktivieren()	Sperrt alle Buttons (nach Spielende)

Farbschema (Catppuccin Mocha):

- Hintergrund: #1E1E2E | Button: #313244 | Hover: #45475A
- Spieler X: #F38BA8 (Rosa) | Spieler O: #89DCEB (Hellblau)

4.4 GameController

Implementiert ActionListener. Wertet den ActionCommand der Buttons aus und koordiniert Logik und GUI.

```
// ActionCommand-Format: "zeile,spalte"
String[] teile = event.getActionCommand().split(",");
int zeile  = Integer.parseInt(teile[0]);
int spalte = Integer.parseInt(teile[1]);
```

4.5 GameWindow

Hauptfenster (extends JFrame) mit vier Bereichen:

- Kopfzeile: Spieltitel auf dunklem Hintergrund
- Mitte: Spielfeld-Wrapper mit Innenabstand
- Fusszeile: Statuslabel + Neustart-Button
- Dialoge: JOptionPane nach Spielende

Das Fenster ist auf 480×580 Pixel fixiert (setResizable(false)) und wird beim Start auf dem Bildschirm zentriert (setLocationRelativeTo(null)).

5. Spielablauf

5.1 Initialisierung

8. Main.java ruft SwingUtilities.invokeLater(() -> new GameWindow()) auf.
9. GameWindow erstellt GameBoard, GameLogic und GameController.
10. GameController registriert sich als ActionListener an den 9 Buttons.
11. Das Fenster erscheint zentriert auf dem Bildschirm, Spieler X beginnt.

5.2 Spielzug

12. Spieler klickt auf ein leeres Feld.
13. GameLogic prüft den Zug (Feld leer? Spiel nicht beendet?).
14. Bei gültigem Zug: Symbol wird gesetzt, Zelle gesperrt, Gewinnerprüfung erfolgt.
15. Statuslabel wird aktualisiert; bei Spielende erscheint ein Dialog.

5.3 Gewinnerprüfung

Die Methode pruefeGewinner() durchläuft nach jedem Zug:

- 3 Zeilen (jeweils alle 3 Zellen gleich)
- 3 Spalten (jeweils alle 3 Zellen gleich)
- Hauptdiagonale ([0,0] [1,1] [2,2])
- Nebendiagonale ([0,2] [1,1] [2,0])

Sind alle 9 Felder belegt und kein Symbol hat gewonnen, meldet istUnentschieden() true.

5.4 Neustart

Über den "Neues Spiel"-Button oder den Ja-Button im Endscreen wird controller.neuesSpiel() aufgerufen:

- GameLogic.spielZuruecksetzen() leert das interne Brett.
- GameBoard.felderZuruecksetzen() aktiviert alle Buttons und entfernt Symbole.
- Das Statuslabel zeigt wieder "Spieler X ist dran".

6. Kompilierung & Start

6.1 Voraussetzungen

- Java Development Kit (JDK) 17 oder neuer
- Kein Build-Tool erforderlich (kein Maven/Gradle)
- Kein Internet / keine externen Bibliotheken

6.2 Terminal-Befehle

Alle fünf .java-Dateien müssen im selben Verzeichnis liegen:

```
# Schritt 1: In den Projektordner wechseln
cd TicTacToe/

# Schritt 2: Alle Quelldateien kompilieren
javac *.java

# Schritt 3: Programm starten
java Main
```

Hinweis zu Java-Versionen

Das Projekt wurde mit Java 21 LTS entwickelt und getestet.

Es ist abwärtskompatibel bis Java 17 (keine neueren Sprachfeatures).

Ältere Versionen (Java 8-16) werden nicht offiziell unterstützt.

6.3 Bekannte Einschränkungen

- Kein KI-Gegner: Das Spiel erfordert zwei menschliche Spieler.
- Kein Highscore-Speicher: Spielstände werden nicht persistiert.
- Fenster nicht skalierbar: Feste Größe von 480×580 Pixel.

7. Erweiterungsmöglichkeiten

Das modulare Design erleichtert die spätere Erweiterung des Projekts erheblich. Da Logik und GUI vollständig getrennt sind, können Verbesserungen isoliert eingeführt werden.

Erweiterung	Betroffene Klasse(n)	Aufwand
KI-Gegner (Minimax-Algorithmus)	GameLogic + GameController	Hoch
Spielstand-Anzeige (Siege/Niederlagen)	GameWindow + GameController	Niedrig
Netzwerkspiel (Client-Server)	Neue Klasse NetworkHandler	Sehr hoch

Erweiterung	Betroffene Klasse(n)	Aufwand
Einstellungen (Spielfeldgröße 4×4)	GameLogic + GameBoard	Mittel
Sound-Effekte bei Zügen	Neue Klasse SoundManager	Niedrig
Animation (Gewinnlinie hervorheben)	GameBoard	Mittel

7.1 KI-Implementierung (Skizze)

Ein Minimax-Algorithmus könnte direkt in GameLogic eingebettet werden, da dort das komplette Spielfeld verfügbar ist:

```
// In GameLogic – neuer KI-Zug
public int[] berechneKIZug() {
    int bestesBewertung = Integer.MIN_VALUE;
    int[] besterZug = {-1, -1};
    for (int z = 0; z < 3; z++) {
        for (int s = 0; s < 3; s++) {
            if (brett[z][s] == PlayerSymbol.LEER) {
                brett[z][s] = PlayerSymbol.O; // KI spielt O
                int score = minimax(false, 0);
                brett[z][s] = PlayerSymbol.LEER;
                if (score > bestesBewertung) {
                    bestesBewertung = score;
                    besterZug = new int[]{z, s};
                }
            }
        }
    }
    return besterZug;
}
```

8. Codequalität & Konventionen

8.1 Kommentarsprache

Alle Kommentare (JavaDoc und Inline) sind auf Deutsch verfasst. Bezeichner (Klassen, Methoden, Variablen) folgen den üblichen Java-Konventionen in Englisch, um maximale IDE-Kompatibilität zu gewährleisten.

8.2 JavaDoc

Alle öffentlichen Klassen und Methoden sind mit JavaDoc-Kommentaren versehen. Jede Klasse enthält:

- Eine einleitende Beschreibung der Verantwortlichkeiten
- @param-Tags für alle Parameter
- @return-Tags für Rückgabewerte
- @throws-Tags wo relevant

8.3 OOP-Prinzipien

Prinzip	Umsetzung im Projekt
Single Responsibility	Jede Klasse hat genau eine Aufgabe
Separation of Concerns	GameLogic kennt kein Swing; GameBoard kennt keine Regeln
Kapselung	Alle Felder private, Zugriff nur über Methoden
Polymorphismus	ActionListener-Interface ermöglicht flexible Ereignisverarbeitung

9. Vollständige Datei-Übersicht

Alle fünf Quelldateien in der Übersicht mit Zeilenzahl und Schlüsselelementen:

Datei	ca. Zeilen	Schlüsselelemente
Main.java	~35	main(), SwingUtilities.invokeLater
PlayerSymbol.java	~60	Enum-Konstanten, andererSpieler(), getAnzeige()
GameLogic.java	~130	brett[][], zugAusfuehren(), pruefeGewinner()
GameBoard.java	~140	JButton[][], MouseAdapter, Farbkonstanten
GameController.java	~70	actionPerformed(), neuesSpiel()
GameWindow.java	~200	JFrame, JLabel, BorderLayout, JOptionPane

Ende der Dokumentation • Tic-Tac-Toe v1.0 • Mai 2026